Assignment No. 1

```
class SimpleReflexVacuumAgent:
    def __init__(self):
        self.location = 'A'

    def sense_and_act(self, status):
        if status == 'Dirty':
            return 'Suck'
        elif self.location == 'A':
            return 'Right'
        elif self.location == 'B':
            return 'Left'

    def update_location(self, action):
        if action == 'Right':
            self.location = 'B'
        elif action == 'Left':
            self.location = 'A'

class VacuumEnvironment:
    def __init__(self, state_dict):
        self.states = state_dict

    def run_simulation(self, agent, steps=4):
        print(f"Initial Environment State: {self.states}")
        print(f"Agent starts at Location: {agent.location}\n")

        for step in range(1, steps + 1):
            current_status = self.states[agent.location]
            action = agent.sense_and_act(current_status)
            print(f"Step {step}: At {agent.location}, Status: {current_status} -> Action:
{action}")

            if action == 'Suck':
                self.states[agent.location] = 'Clean'
            else:
                agent.update_location(action)
    print(f"\nFinal Environment State: {self.states}")

initial_state = {'A': 'Dirty', 'B': 'Dirty'}
my_agent = SimpleReflexVacuumAgent()
sim = VacuumEnvironment(initial_state)
sim.run_simulation(my_agent, steps=4)
```

Assignment No.2

```
import heapq
GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def heuristic(state):
    distance = 0
    for i, value in enumerate(state):
        if value == 0:
            continue
        current_row = i // 3
        current_col = i % 3
        goal_index = GOAL.index(value)
        goal_row = goal_index // 3
        goal_col = goal_index % 3
        distance += abs(current_row - goal_row)
        distance += abs(current_col - goal_col)
    return distance

def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    row = blank // 3
    col = blank % 3
    moves = [
        (-1, 0),   # Up
        (1, 0),    # Down
        (0, -1),   # Left
        (0, 1)     # Right
    ]
    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank = new_row * 3 + new_col
            new_state = list(state)
            new_state[blank], new_state[new_blank] = (
                new_state[new_blank],
                new_state[blank]
            )
            neighbors.append(tuple(new_state))
    return neighbors
```

```
def a_star(start):
    priority_queue = []
    h = heuristic(start)
    heapq.heappush(priority_queue, (h, 0, start, []))
    visited = set()
    while priority_queue:
        f, g, state, path = heapq.heappop(priority_queue)
        if state in visited:
            continue
        visited.add(state)
        path = path + [state]
        if state == GOAL:
            return path
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                new_g = g + 1
                new_h = heuristic(neighbor)
                new_f = new_g + new_h
                heapq.heappush(
                    priority_queue,
                    (new_f, new_g, neighbor, path)
                )
    return None

def print_state(state):
    for i in range(0, 9, 3):
        row = state[i:i + 3]
        print(" ".join("_" if x == 0 else str(x) for x in row))
    print()
print("========== A* Search for 8-Puzzle ==========")

start = tuple(
    map(
        int,
        input("Enter initial state (use 0 for blank): ").split()
    )
)
solution = a_star(start)
if solution:
    print("\nSolution Found!")
    print("Number of moves:", len(solution) - 1)
    print("\nSolution Path:")
    for step, state in enumerate(solution):
        print("Step", step)
```

```
        print_state(state)
else:
    print("No solution found.")
```

Assignment No. 3

```
def hanoi_state_space(n):
    source = list(range(n, 0, -1))
    auxiliary = []
    destination = []
    pegs = [source, auxiliary, destination]
    names = ["Source", "Auxiliary", "Destination"]
    moves = []

    def move_disk(src, dst):
        if not pegs[src]:
            return False
        if pegs[dst] and pegs[dst][-1] < pegs[src][-1]:
            return False
        disk = pegs[src].pop()
        pegs[dst].append(disk)
        moves.append(f"Move disk {disk} from {names[src]} to {names[dst]}")
        return True

    def solve(k, src, aux, dst):
        if k == 0:
            return
        solve(k - 1, src, dst, aux)
        move_disk(src, dst)
        solve(k - 1, aux, src, dst)
    print("========== Tower of Hanoi ==========")
    print("Initial State:", pegs)
    solve(n, 0, 1, 2)
    print("\nSolution Path:")
    for i, move in enumerate(moves, 1):
        print(f"{i}. {move}")
    print("\nGoal State:", pegs)
    print("Total Moves:", len(moves))
n = int(input("Enter number of disks: "))
hanoi_state_space(n)
```

Assignment No. 4

```
def alpha_beta(node, alpha, beta, maximizing_player):
    if not isinstance(node, list):
        return node
    if maximizing_player:
        value = float('-inf')
        for child in node:
            value = max(value, alpha_beta(child, alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta:
                print("Pruning at MAX node")
                break
        return value
    else:
        value = float('inf')
        for child in node:
            value = min(value, alpha_beta(child, alpha, beta, True))
            beta = min(beta, value)
            if alpha >= beta:
                print("Pruning at MIN node")
                break
        return value
print("Enter the game tree in Python list format.")
print("Example: [[[3,5],[6,9]],[[1,2],[0,-1]]]")
game_tree = eval(input("Enter game tree: "))
alpha = float('-inf')
beta = float('inf')
optimal_value = alpha_beta(game_tree, alpha, beta, True)
print("\nOptimal Game Value =", optimal_value)
```

Assignment No. 5

```
import re
rules = [
    (r"hello|hi|hey",
    "Hello! How can I help you?"),
    (r"how are you",
     "I am fine. Thank you for asking!"),
    (r"what is your name|your name",
     "My name is SimpleBot."),
```

```
    (r"who are you",
     "I am a simple rule-based chatbot."),
    (r"what can you do|help",
     "I can answer simple questions using predefined rules."),
    (r"what is artificial intelligence|define ai",
     "Artificial Intelligence is the simulation of human intelligence in machines."),
    (r"thank you|thanks",
     "You are welcome!"),
    (r"bye|goodbye|exit",
     "Goodbye! Have a nice day.")
]
def chatbot_response(user_input):
    user_input = user_input.lower().strip()
    for pattern, response in rules:
        if re.search(pattern, user_input):
            return response
    return "Sorry, I don't understand that."
print("====================================")
print("       SIMPLE RULE-BASED CHATBOT")
print("====================================")
print("Type 'bye' to exit the chatbot.\n")
while True:
    user_input = input("You: ")
    response = chatbot_response(user_input)
    print("Bot:", response)
    if re.search(r"bye|goodbye|exit", user_input.lower()):
        break
```

Assignment No. 6

```
N = 8
def print_board(board):
    for row in board:
        print(" ".join("Q" if cell == 1 else "." for cell in row))

def is_safe(board, row, col):
    for i in range(row):
        if board[i][col] == 1:
            return False
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
```

```
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    i, j = row - 1, col + 1
    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1
    return True

def solve_queens(board, row):
    if row == N:
        return True
    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            if solve_queens(board, row + 1):
                return True
            board[row][col] = 0
    return False
def main():
    board = [[0 for _ in range(N)] for _ in range(N)]
    if solve_queens(board, 0):
        print("Solution for 8-Queens Problem:")
        print_board(board)
    else:
        print("No solution exists.")
if __name__ == "__main__":
    main()
```

Assignment No. 7

```
from collections import deque
def bfs_8_puzzle(start, goal):
    start = tuple(start)
    goal = tuple(goal)
    queue = deque([start])
    parent = {start: None}
    action_taken = {start: None}
```

```
    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }
    while queue:
        state = queue.popleft()
        if state == goal:
            path = []
            while parent[state] is not None:
                path.append((action_taken[state], state))
                state = parent[state]
            return path[::-1]
        zero = state.index(0)
        row, col = divmod(zero, 3)
        for action, change in moves.items():
            if action == "Up" and row == 0:
                continue
            if action == "Down" and row == 2:
                continue
            if action == "Left" and col == 0:
                continue
            if action == "Right" and col == 2:
                continue
            new_zero = zero + change
            new_state = list(state)
            new_state[zero], new_state[new_zero] = (
                new_state[new_zero],
                new_state[zero]
            )
            new_state = tuple(new_state)
            if new_state not in parent:
                parent[new_state] = state
                action_taken[new_state] = action
                queue.append(new_state)
    return None
start = [1, 2, 3, 4, 0, 6, 7, 5, 8]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]
solution = bfs_8_puzzle(start, goal)
print("========== 8 Puzzle using BFS ==========")
if solution is not None:
    print("Solution found!")
```

```
    print("Number of moves:", len(solution))
    for i, (action, state) in enumerate(solution, 1):
        print("\nMove", i, ":", action)
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
else:
    print("No solution found.")
```

Assignment No. 8

```
def goal_stack_planning():
    state = {
        "OnTable(A)", "OnTable(B)", "OnTable(C)",
        "Clear(A)", "Clear(B)", "Clear(C)", "HandEmpty"
    }
    goals = ["On(A,B)", "On(B,C)"]
    actions = []
    if "On(A,B)" not in state:
        actions.append("PickUp(A)")
        state.remove("OnTable(A)")
        state.remove("Clear(A)")
        state.remove("HandEmpty")
        state.add("Holding(A)")
        actions.append("Stack(A,B)")
        state.remove("Holding(A)")
        state.remove("Clear(B)")
        state.add("On(A,B)")
        state.add("Clear(A)")
        state.add("HandEmpty")
    if "On(B,C)" not in state:
        actions.append("UnStack(A,B)")
        state.remove("On(A,B)")
        state.remove("Clear(A)")
        state.remove("HandEmpty")
        state.add("Holding(A)")
        state.add("Clear(B)")
        actions.append("PutDown(A)")
        state.remove("Holding(A)")
        state.add("OnTable(A)")
        state.add("Clear(A)")
        state.add("HandEmpty")
```

```
        actions.append("PickUp(B)")
        state.remove("OnTable(B)")
        state.remove("Clear(B)")
        state.remove("HandEmpty")
        state.add("Holding(B)")
        actions.append("Stack(B,C)")
        state.remove("Holding(B)")
        state.remove("Clear(C)")
        state.add("On(B,C)")
        state.add("Clear(B)")
        state.add("HandEmpty")
    print("========== Goal Stack Planning ==========")
    print("\nInitial State:")
    print("{OnTable(A), OnTable(B), OnTable(C), Clear(A), Clear(B), Clear(C),
HandEmpty}")
    print("\nGoal State:")
    print("{On(A,B), On(B,C), OnTable(C), Clear(A), HandEmpty}")
    print("\nPlan Generated:")
    for i, action in enumerate(actions, 1):
        print(f"Step {i}: {action}")
goal_stack_planning()
```